Git Guide (Alex Murphy)

1. Init

‘git init’ is used to create a new empty git repository. An initial branch without any commits will be created. Running the command in an existing repository will not overwrite things that are already there.

```
c00335025@CD-A307-34 MINGW64 /c/Users/c00335025/Desktop/Git-Guide (main)
$ git init
Reinitialized existing Git repository in C:/Users/c00335025/Desktop/Git-Guide/.git/
```

2. Add

‘git add’ adds contents or new or changed files to the index (what you use to prepare the contents of the next commit). When ‘git commit’ is ran without any other arguments, it will only commit staged changes.

```
c00335025@CD-A307-34 MINGW64 /c/Users/c00335025/Desktop/Git-Guide (main)
$ git add README.md
```

3. Status

‘git status’ shows the working tree status. It displays paths that have differences between the index file and the HEAD commit, paths that have differences between the working tree and the index, and paths in the working tree that aren’t tracked by Git.

```
c00335025@CD-A003-54 MINGW64 /m/Git-Guide (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ~$guide.docx

nothing added to commit but untracked files present (use "git add" to track)
```

4. Commit

‘git commit’ records changes to the selected repository. It creates a new commit containing the current contents of the index and the given log message describing the changes.

```
c00335025@CD-A003-54 MINGW64 /m/Git-Guide (main)
$ git commit
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ~$guide.docx

nothing added to commit but untracked files present (use "git add" to track)
```

5. Remote

‘git remote’ manages the set of repositories whose branches you track.

```
c00335025@CD-A003-54 MINGW64 /m/Git-Guide (main)
$ git remote
origin
```

6. Branch

‘git branch’ is used to list, create or delete branches.

```
c00335025@CD-A003-54 MINGW64 /m/Git-Guide (main)
$ git branch
* main
```

7. Checkout

‘git checkout’ switches branches or restores working tree files.

```
c00335025@CD-A003-54 MINGW64 /m/Git-Guide (main)
$ git checkout
Your branch is up to date with 'origin/main'.
```

8. Merge

‘git merge’ joins two or more development histories together. It incorporates changes from the named commits into the current branch.

```
c00335025@CD-A003-54 MINGW64 /m/Git-Guide (main)
$ git merge
Already up to date.
```

9. Log

‘git log’ shows the commit logs. Lists commits that are reachable by following the parent links from the given commit(s)

```
c00335025@CD-A003-54 MINGW64 /m/Git-Guide (main)
$ git log
commit 604d4398e0dcadd3b2d299be73438f846fff0d27 (HEAD -> main)
Author: (Student  C00335025) Alex Murphy <C00335025@setu.ie>
Date:   Tue Oct 20 13:50:45 2026 +0100

    8. merge

commit 4d4c4c01e6a017ef12b0674ad9d1f36bd35661cd
Author: (Student  C00335025) Alex Murphy <C00335025@setu.ie>
Date:   Tue Oct 20 13:40:25 2026 +0100

    i have done up to 7 without saving sorry

commit f796731efd422c3baf53f795d0381e726c2e77fd (origin/main)
Author: (Student  C00335025) Alex Murphy <C00335025@setu.ie>
Date:   Tue Oct 20 13:10:59 2026 +0100

    add in screenshots
```

10. Push

‘git push’ updates remote refs along with associated objects. It updates one or more branches, tags, or other references in one or more remote repositories from your local repository, and sends all necessary data that isn’t already on the remote.

```
c00335025@CD-A003-54 MINGW64 /m/Git-Guide (main)
$ git push
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 20 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 106.89 KiB | 5.63 MiB/s, done.
Total 10 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), completed with 1 local object.
To https://github.com/Alex-Murphy-1/Git-Guide.git
   f796731..f01be75  main -> main
```

Command 1. Init
‘git init’ is used to create a new empty git repository. An initial branch without any commits will be created. Running the command in an existing repository will not overwrite things that are already there.

Command 2. Add
‘git add’ adds contents or new or changed files to the index (what you use to prepare the contents of the next commit). When ‘git commit’ is ran without any other arguments, it will only commit staged changes.

Command 3. Status
‘git status’ shows the working tree status. It displays paths that have differences between the index file and the HEAD commit, paths that have differences between the working tree and the index, and paths in the working tree that aren’t tracked by Git

Command 4. Commit
‘git commit’ records changes to the selected repository. It creates a new commit containing the current contents of the index and the given log message describing the changes.

Command 5. Remote
‘git remote’ manages the set of repositories whose branches you track.

Command 6. Branch
‘git branch’ is used to list, create or delete branches.

Command 7. Checkout
‘git checkout’ switches branches or restores working tree files.

Command 8. Merge
‘git merge’ joins two or more development histories together. It incorporates changes from the named commits into the current branch.

Command 9. Log
‘git log’ shows the commit logs. Lists commits that are reachable by following the parent links from the given commit(s)

Command 10. Push
‘git push’ updates remote refs along with associated objects. It updates one or more branches, tags, or other references in one or more remote repositories from your local repository, and sends all necessary data that isn’t already on the remote.